VITyarthi| Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
POORVA JAISWAL	24BCE11486
Slot	**Date of Submission**
B22	30/03/2026

Part A — Origin and Philosophy

A1. The Problem Git Was Created to Solve

The Situation Before Git:
Before Git came into the picture, managing code for large projects was quite difficult. Most teams relied on centralized version control systems like CVS (Concurrent Versions System) or Subversion (SVN). These systems depended completely on a central server, which created several problems.

In real-world use, this caused issues such as:

- If the server stopped working, developers couldn't commit their code
- Creating branches and merging code was slow and often led to errors
- Working without internet access was nearly impossible
- Performance dropped significantly in large projects with many contributors

A major example was the Linux kernel project. It had thousands of developers contributing from different parts of the world, which made these issues even more noticeable.

The BitKeeper Crisis:

Between 2002 and 2005, the Linux team used a tool called BitKeeper. It worked in a distributed way, meaning each developer had a full copy of the project. This solved many earlier problems.

However, things changed in 2005. A developer tried to study how BitKeeper worked internally, which violated its license. As a result, the creator of BitKeeper removed free access for the Linux community.

Suddenly, one of the most important open-source projects lost its version control system.

Linus Takes Action:

Linus Torvalds decided to create a new system. He did not like existing tools because of their centralized nature. His goal was simple: avoid all the problems that older systems had.

Even the name “Git” was chosen casually, as a self-critical joke.

Why Open Source?

Git was made open source for both practical and philosophical reasons:

1. The BitKeeper situation showed the risks of relying on proprietary tools
2. The Linux community needed a reliable solution
3. Linus believed in openness and collaboration

Real-World Insights

- **Speed was important:** Git was designed to handle large projects quickly. Tasks that earlier took minutes could now be done in seconds.
- **No single point of failure:** Every developer has a complete copy of the project, so work doesn’t stop if one system fails.
- **Less conflict over control:** Developers can work independently and share their work based on quality rather than permissions.

A2. The License — What It Actually Says

Git's License: GPL Version 2
Git is released under the GNU General Public License Version 2 (GPL v2). This license is not just legal text, but a clear idea about how software should be shared and used.

The Four Freedoms of Free Software

The Free Software Foundation defines four important freedoms:

- **Freedom 0:** You can use the software for any purpose
- **Freedom 1:** You can study and modify the code
- **Freedom 2:** You can share copies with others
- **Freedom 3:** You can distribute modified versions

Git provides all these freedoms to users.

GPL vs MIT License
There is an important difference between GPL and MIT licenses:

- Both allow usage and modification
- GPL requires sharing your changes if you distribute the software
- MIT allows keeping modifications private

For example, if someone improves Git:

- Under GPL, they must share the changes if they distribute it
- Under MIT, they could keep it private and sell it

"Free as in Beer" vs "Free as in Freedom"

- **Free as in beer:** No cost
- **Free as in freedom:** Full control

Git provides both. It is free to use and also gives complete control over the software.

Which License to Choose?

- GPL is better when you want your work to remain open
- MIT is better when you want wider adoption without restrictions

For a tool like Git, GPL is suitable because it prevents companies from making it closed-source in the future.

A3. The Ethics of Open Source

Should All Software Be Open Source?
This question has different viewpoints.

Arguments in favor:

- Transparency improves security
- Public systems should be open for verification
- Users are not dependent on one company
- Innovation becomes faster through collaboration

Arguments against:

- Some companies rely on secrecy for business
- Not all software needs community involvement
- Open projects may lack proper maintenance
- Developers need financial support

Is It Ethical for Companies to Profit?
Large companies often use open-source software to build profitable products.

One view is that companies contribute back through funding, tools, and development support.

Another view is that the benefit they receive is much larger than what individual contributors get.

Open Source Contribution
Software development builds on previous work. Every system depends on earlier technologies created by others. Open source makes this process transparent and collaborative.

Final Thought
Open source works best for foundational tools like operating systems, version control, and databases. The key issue is not usage, but contribution. Companies benefiting from open source should also support its growth.

Part B — Linux Footprint

How Git Works on a Linux System
When Git is installed on a Linux system like Ubuntu, it does not exist as just one single file or program. Instead, it is distributed across different directories in the system. This follows a standard structure used by Linux, known as the Filesystem Hierarchy Standard. Understanding this helps in knowing how software is organized and managed inside the operating system.

Installation Method
Git is usually installed using the system's package manager. In Ubuntu, this is done using APT (Advanced Package Tool). The installation process is simple, but internally it performs several tasks:

- It downloads the required files from official repositories
- It installs all necessary dependencies automatically
- It places different components of Git in their correct system locations
- It registers Git in the system so it can receive future updates

Although it is also possible to install Git by compiling its source code manually, this is generally not required unless a user needs the latest experimental features.

Where Git Installs

Location	What's There	Why It Matters
/usr/bin/git	The main Git executable	This is what runs when you type git
/usr/lib/git-core/	Git subcommands	Commands like git-commit, git-push live here
/usr/share/git-core/	Templates and default files	Used when initializing new repos
/usr/share/doc/git/	Documentation	Man pages and help files
~/.gitconfig	User configuration	Your name, email, preferences
.git/ (in any repo)	Repository data	The actual version history

User Permissions and Security

Git runs under the current user account, not as an administrator. This is important from a security perspective:

- Git can only access files that the user has permission to access
- It cannot modify system-level files without proper authorization
- If there is an issue or bug, the impact is limited to the user's environment

When a user performs actions like committing code, Git records the user's name and email, which are defined in the configuration file.

This setup is especially useful in shared systems, where multiple users work independently without interfering with each other's work.

Update Mechanism

Git is updated through the operating system's package management system. The update process generally follows these steps:

1. Developers release updates or security fixes
2. Package maintainers review and test these updates
3. Updated packages are added to the system repositories
4. Users receive updates through standard system upgrade commands

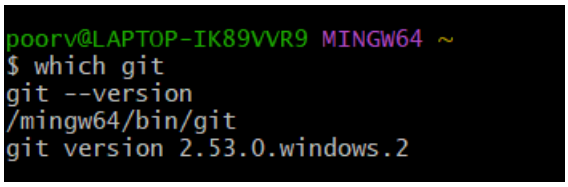
This process ensures that Git remains stable and secure while integrating smoothly with the operating system.

Service Behavior

Git is not a background service. It does not run continuously in the system. Instead, it works as a command-line tool that executes only when the user runs a command.

In some advanced use cases, Git can be used as a server (for example, for hosting repositories). In such cases, additional services may be configured and managed separately. However, for most users, Git functions as a simple and efficient tool that runs only when needed.

Screenshot 1- Locating Git Binary in System and Checking Installed Git Version



Screenshot 2- Viewing Git User Configuration File

```
poorv@LAPTOP-IK89VVR9 MINGW64 ~
$ ls -la ~/.gitconfig
-rw-r--r-- 1 poorv 197609 59 Sep  3  2025 /c/Users/poorv/.gitconfig
```

Screenshot 3- Displaying Git Configuration Details

```
poorv@LAPTOP-IK89VVR9 MINGW64 ~
$ git config --global user.name "Student Name"
git config --global user.email "student@example.com"

poorv@LAPTOP-IK89VVR9 MINGW64 ~
$ cat ~/.gitconfig
[user]
        email = student@example.com
        name = Student Name
```

Screenshot 4- Showing Git Installation Path

```
poorv@LAPTOP-IK89VVR9 MINGW64 ~
$ where git
C:\Program Files\Git\mingw64\bin\git.exe
C:\Program Files\Git\cmd\git.exe
```

Part C — The FOSS Ecosystem

The World Around Git

Git is not just a standalone tool. It acts as the base for a much larger ecosystem of platforms, tools, and communities that together support modern software development. Today, almost every major project depends on Git in some way, directly or indirectly.

What Git Depends On

One of the reasons Git is widely used is because it has very few dependencies. This makes it lightweight and easy to run on different systems. Some of its main dependencies include:

- A C compiler, since Git is written in C for better performance
- zlib, which is used for compressing data efficiently
- OpenSSL or libcurl, which help in secure communication over networks
- POSIX shell, as some Git commands are implemented using shell scripts

- Perl (optional), used for certain older or additional tools

This minimal requirement ensures that Git can run on a wide range of devices, from small systems to powerful servers.

Tools Built on Top of Git
The real strength of Git comes from the tools and platforms built around it. These tools extend Git's functionality and make it easier to use in real-world development.

Hosting Platforms:
These platforms allow developers to store and manage their code online:
- GitHub
- GitLab
- Bitbucket
- Self-hosted options like Gitea or Gogs

CI/CD Tools:
These tools automate testing and deployment processes:
- GitHub Actions
- GitLab CI
- Jenkins
- CircleCI and Travis CI

Code Review Systems:
Git makes it easy to review code changes before merging:
- Pull requests and merge requests
- Tools like Gerrit for structured review processes

Development Environments:
Most modern code editors and IDEs support Git directly:
- Visual Studio Code
- JetBrains IDEs such as IntelliJ and PyCharm
- Editors like Vim and Emacs through plugins

How Git Fits in Web Development
Git plays an important role in modern web development workflows. It is often used as the central system for managing and deploying code.

A common workflow looks like this:
- Developers write and store code in a Git repository
- Changes are pushed to a platform like GitHub or GitLab
- Automated tools test the code and deploy it

- Servers update the application with the latest version

This approach is often called “GitOps”, where Git acts as the single source of truth for the system.

The Community Behind Git
Git is maintained by a global community rather than a single company. Its development is guided by experienced maintainers and contributions from developers around the world.

Key community resources include:
- Official documentation websites
- Mailing lists where development discussions happen
- Online platforms for issue tracking and collaboration
- Developer forums and Q&A sites

The project follows a structured model where a main maintainer oversees contributions, ensuring stability and quality.

Projects That Use Git
Git is used by almost every major open-source and software project today. Some examples include:
- Operating systems like Linux
- Programming languages such as Python and JavaScript environments
- Frontend frameworks like React and Angular
- Tools like Docker and Kubernetes
- Applications such as web browsers and code editors

This wide usage creates a strong network effect. Since most developers already use Git, new projects naturally adopt it as well.

Real-World Insights
1. **Git as infrastructure:**
   Git has become so essential that it is often taken for granted. It plays a role similar to basic infrastructure in software development.
2. **Ecosystem advantage:**
   Git alone is powerful, but when combined with platforms, automation tools, and editors, it becomes a complete development system.
3. **Platform independence:**
   Different platforms may offer different features, but since all of them are built on Git, users can switch between them without losing their work.

Part D — Open Source vs Proprietary Comparison

Git vs Proprietary Version Control Systems
To understand the importance of Git, it helps to compare it with proprietary version control systems. One well-known example is Perforce Helix Core, which is commonly used in large companies and game development.

Comparison Table

Dimension	Git (Open Source)	Perforce Helix Core (Proprietary)
Cost	Free for all uses	Free for up to 5 users; enterprise licenses cost thousands per user annually
Security (code audit)	Anyone can audit the source code; vulnerabilities are found and fixed publicly	Source code is secret; you trust the vendor's security claims
Support	Community forums, Stack Overflow, paid support from vendors like GitHub Enterprise	Direct vendor support included with license
Reliability	Battle-tested by millions of projects; no single point of failure due to distributed design	Centralized server; if it goes down, work stops
Freedom to modify	Complete freedom to fork, modify, and customize	You use what they give you; modifications require negotiation
Large binary files	Weak by default; requires Git LFS extension	Excellent; designed for game assets and large files
Learning curve	Steeper; concepts like rebasing confuse beginners	Simpler model for basic operations

Community vs corporate control	Community-governed; no single company controls direction	Perforce Inc. controls everything
Offline work	Full functionality offline; you have complete history	Limited; most operations require server connection
Ecosystem	Massive; GitHub, GitLab, thousands of integrations	Smaller; mostly enterprise tooling

Detailed Analysis

Cost

Git removes financial barriers. Anyone can use it, whether they are students, startups, or large organizations. This makes it highly accessible.

However, even though Git itself is free, there can still be indirect costs such as:

- Training team members
- Using enterprise hosting services
- Managing infrastructure for large teams

Proprietary tools, on the other hand, involve direct licensing costs, which can be expensive for large teams.

Security

At first, open-source software may seem less secure because its code is publicly available. But in practice, this transparency improves security.

- Bugs and vulnerabilities can be identified by a large number of developers
- Fixes are openly reviewed and verified

In proprietary systems:

- Users must trust the company's internal security processes
- There is limited visibility into how issues are handled

However, open source also depends on active contributors. If a project is not well maintained, issues may remain unnoticed.

Support and Reliability

Proprietary tools usually provide dedicated support services. This means:

- Users can contact support teams directly
- Issues are handled professionally and quickly

Git relies mostly on:

- Documentation
- Community forums
- Online discussions

For most users, this is enough. But for critical business systems, dedicated support may be preferred.

Freedom to Modify
This is one of Git's strongest advantages.
- Users can modify Git according to their needs
- Developers can create custom features or workflows
- There are no restrictions on how Git is used

In proprietary systems:
- Users must rely on the features provided
- Customization is limited
- Feature requests depend on the company's decisions

Where Proprietary Tools Perform Better
Despite Git's advantages, proprietary tools like Perforce perform better in certain areas.

For example:
- Handling very large binary files such as videos, game assets, or design files
- Managing extremely large repositories in industries like gaming and film production

Git can handle such cases with extensions, but it is not as efficient by default.

Overall Comparison
For most software development use cases, Git is the preferred option because:
- It is free and widely accessible
- It has a large and active ecosystem
- It provides flexibility and control
- It is supported by a strong global community
- It avoids dependency on a single vendor

Proprietary tools are more suitable in specialized scenarios where specific features are required.

Final Verdict
Git is the industry standard for version control today. Its distributed design, flexibility, and strong ecosystem make it suitable for almost all types of software development.

Unless there are very specific requirements, Git is the default and most practical choice.

Learning and Contribution Perspective
Using Git is not just important for development, but also for learning and career growth.

- It helps in understanding real-world collaboration
- It is used in almost every software company
- Contributing to Git or related projects can improve technical skills

Even small contributions, such as improving documentation, can provide valuable experience in working with large codebases.